Guía práctica de remasterización a 4K con IA: Antena 3 Noticias (1992)

Guía técnica: alternativas comparadas, estrategia elegida, pipeline completo y código

Javier Ortiz Zamora · Universidad de Alicante (UA)
Material: *Antena 3 Noticias*, emisión del 28-09-1992 (y tramo del 12-09-1991)
Versión de trabajo · 2 de noviembre de 2026
Equipo: Apple M5, cálculo en GPU Metal (MPS). Sin pago por servicios ni APIs de pago.

1. Objetivo y alcance

El objetivo es llevar un informativo de televisión emitido en PAL (640×480, 25 fotogramas por segundo) a una versión de 3840×2160 con 50 fotogramas por segundo, usando solo software libre o gratuito y procesamiento en un equipo local. Se busca además que el resultado sea fiel al original: el sistema no debe inventar rostros, letras ni detalles que no existen en la fuente.

Este documento sirve también como guía práctica: recoge las alternativas técnicas probadas, la razón de la estrategia final, el pipeline completo y el código que lo implementa. Está pensado como base para una futura nota técnica o artículo; por ahora no incluye la evaluación objetiva de calidad (sección 7.3).

2. Material de partida

Las emisiones PAL antiguas tienen tres limitaciones que condicionan cualquier restauración:

- **Resolución baja:** 640×480 en el informativo de 1992 (aprox. 0,3 Mpx por fotograma, frente a 8,3 Mpx de 4K).
- **Entrelazado perdido:** la copia disponible es progresiva a 25 fps. Los campos originales (50 semicuadros por segundo) ya no existen en el archivo, así que **no se pueden recuperar 50 fps reales**. La cadencia de 50 fps se obtiene por interpolación de fotogramas, no por recuperación de información.
- **Compresión y ruido:** artefactos de codificación, bandas negras laterales (imagen 4:3 dentro de un lienzo mayor) y granulado en planos de baja luz.

El análisis con `idet` y la detección de bandas con `cropdetect` se automatizan en el script (funciones `detectar_entrelazado` y `detectar_recorte`), de modo que el mismo proceso sirve para la intro de 1989, el informativo de 1991 y el de 1992.

3. Alternativas comparadas

Se evaluaron cuatro métodos sobre los mismos tramos de vídeo. Los tiempos son medidos en el equipo y se registran automáticamente en `registro_tiempos.csv` (minutos de cálculo por minuto de vídeo, en la columna *min_calculo_por_min_video*).

Método	Qué hace	Coste medido (min por min de vídeo)	Observación
A. Lanczos (línea base)	Escalado con FFmpeg, sin IA	0,36–0,45 (típico 0,38)	Referencia sin inventar detalle. Suave, sin cambios de textura.
B. Real-ESRGAN general	Super-resolución ×4 con `realesr-general-x4v3` (mezcla con la variante de denoise al 50 %)	5,8–8,1	Mejora de bordes y textura en letras y paisaje. Efecto notable pero sin rostros tratados.
C. Real-ESRGAN x4plus	Modelo RRDB de mayor capacidad (`RealESRGAN_x4plus`)	96–194	Letras más nítidas, pero **inventa textura** donde no hay información. Sin ganancia visible en la intro de 1989, en la cabecera ni en la cortinilla de 1992.
D. Estrategia zonal (caras)	Método B + CodeFormer sobre los rostros detectados (RetinaFace), con mezcla gradual según el tamaño de la cara	23–36	La mejora más llamativa en primer plano. Problema de partida: en planos lejanos las caras quedaban como "muñecos de cera", por lo que se añadió el control por tamaño (sección 4.2).

3.1. Lectura de la comparación

Las pruebas muestran que el modelo más grande (C) no compensa el coste cuando el contenido no tiene detalle recuperable: en la intro de 1989 se pagó casi cien minutos de cálculo por minuto de vídeo sin ganancia visible, y el rótulo rojo llegó a desbordarse. La ganancia real aparece donde hay una estructura reconocible que el modelo sabe completar, como los rostros. Por eso el método final (D) combina el modelo general (B), que es barato, con un restaurador especializado solo en las zonas donde compensa.

3.2. Por qué no se usó Topaz u otro software de pago

Los programas comerciales de referencia implementan algoritmos del mismo tipo. Para un trabajo de comparación o de docencia, el objetivo es reproducir ese comportamiento con componentes abiertos y medirlo, sin depender de una licencia de pago.

4. Estrategia elegida

4.1. Super-resolución general más restauración de rostros

La super-resolución general (B) se aplica a todo el fotograma. CodeFormer (D) solo actúa sobre los rostros, que se detectan con RetinaFace, se alinean a 512 píxeles, se restauran y se vuelven a pegar sobre el fondo ya escalado. Se usa el parámetro de fidelidad `w = 0,85`, que prioriza la similitud con el original frente a la invención de detalle.

4.2. Control de las "caras de cera"

Cuando la cámara se aleja, la cara ocupa pocos píxeles del original y CodeFormer genera una piel excesivamente lisa. La solución es proporcional al tamaño de la cara *en el original*:

- Menos de **40 px**: no se aplica CodeFormer; queda el resultado de Real-ESRGAN.
- Entre **40 y 90 px**: mezcla gradual, con peso lineal entre 0 y 1.
- Más de **90 px**: CodeFormer completo.

El mapa de mezcla se suaviza con un desenfoque gaussiano para que no aparezca el borde de la cara. El código correspondiente está en el Anexo A.

4.3. Interpolación a 50 fps

RIFE 4.9 genera un fotograma intermedio entre cada par consecutivo. Cuando la diferencia media entre fotogramas supera 0,12 se considera un corte de plano y se repite el fotograma anterior, para no mezclar dos planos distintos. En el informativo de 1992 se detectaron 13 cortes.

5. Arquitectura del pipeline

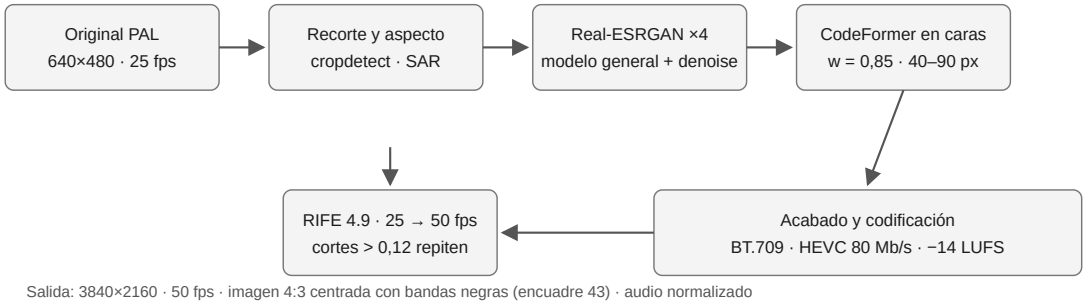


Figura 1. Pipeline final. Todas las etapas se conectan por tuberías (pipes) en memoria, sin fotogramas intermedios en disco.

Las etapas, en orden:

1. **Lectura y análisis.** FFprobe obtiene resolución, SAR, fps y pista de audio. `idet` decide si hay entrelazado; `cropdetect` detecta bandas negras.
2. **Recorte y aspecto.** El contenido se escala con Lanczos a la altura 2160 y se centra en un lienzo 3840×2160 con el aspecto de pantalla correcto (4:3 para PAL).
3. **Super-resolución.** Cada fotograma pasa por Real-ESRGAN en MPS, con troceado en bloques de 256 px para no agotar la memoria de la GPU.
4. **Restauración de caras.** CodeFormer según la sección 4.2.
5. **Interpolación.** RIFE genera los fotogramas intermedios y detecta cortes.
6. **Acabado.** Corrección ligera de contraste, saturación y tono de piel; etiquetado BT.709 y audio normalizado a −14 LUFS con `loudnorm`.

6. Implementación

El proceso se implementa en Python con PyTorch, usando FFmpeg como lector y codificador. Los fotogramas se pasan por tuberías en formato RGB24, sin escribirlos en disco. Se incluyen tres mecanismos de robustez, pensados por la experiencia con el antivirus del equipo y con cortes en mitad del proceso:

- La salida se escribe en un archivo `.parcial` y solo se renombra cuando el proceso termina bien.
- La opción `--saltar-hechos` omite las salidas ya completas, para retomar sin repetir.
- Cada ejecución añade una línea a `registro_tiempos.csv` con tiempos, dispositivo y parámetros, lo que permite reproducir las cifras de este informe.

6.1. Procesamiento principal (`remaster4k.py`, función `procesar`)

```
        vf_in = ",".join(filtros + ["format=bgr24"])
        dec = subprocess.Popen(
            ["ffmpeg", "-hide_banner", "-loglevel", "error"] + tramo
            + ["-i", str(ruta), "-map", "0:v:0", "-vf", vf_in, "-fps_mode", "passthrough",
               "-f", "rawvideo", "-pix_fmt", "bgr24", "-"],
            stdout=subprocess.PIPE, stderr=subprocess.PIPE, bufsize=w * h * 3 * 4)
        W4, H4 = w * 4, h * 4
        enc_cmd = (["ffmpeg", "-hide_banner", "-loglevel", "error", "-y",
                    "-f", "rawvideo", "-pix_fmt", "bgr24", "-s", f"{W4}x{H4}",
                    "-framerate", str(fps_out), "-i", "-"]
                   + tramo + ["-i", str(ruta), "-map", "0:v:0", "-vf", vf_final]
                   + enc + audio_map + ["-shortest", "-movflags", "+faststart", str(salida)])
        encp = subprocess.Popen(enc_cmd, stdin=subprocess.PIPE)
```

6.2. Troceado para la super-resolución (`remaster4k.py`, clase `Escalador`)

```
    def _troceado(self, t):
        torch = self.torch
        _, c, h, w = t.shape
        s, ts, p = 4, self.tile, self.pad
        out = torch.zeros((1, c, h * s, w * s), device=t.device, dtype=t.dtype)
        for y0 in range(0, h, ts):
            for x0 in range(0, w, ts):
                y1, x1 = min(y0 + ts, h), min(x0 + ts, w)
                py0, px0 = max(y0 - p, 0), max(x0 - p, 0)
                py1, px1 = min(y1 + p, h), min(x1 + p, w)
                o = self._infer(t[:, :, py0:py1, px0:px1])
                oy, ox = (y0 - py0) * s, (x0 - px0) * s
                out[:, :, y0 * s:y1 * s, x0 * s:x1 * s] = \
                    o[:, :, oy:oy + (y1 - y0) * s, ox:ox + (x1 - x0) * s]
        return out
```

6.3. Restauración de caras con control por tamaño (`remaster4k.py`, clase `RestauradorCaras`)

```
    def _peso(self, ancho):
        a = (ancho - self.cara_min) / (self.cara_plena - self.cara_min)
        return float(min(max(a, 0.0), 1.0))

    def __call__(self, bgr, fondo_escalado):
        torch, h = self.torch, self.helper
        h.clean_all()
        h.read_image(bgr)
        n = h.get_face_landmarks_5(only_center_face=False, resize=640, eye_dist_threshold=5)
        if n == 0:
            return fondo_escalado
        self.n_caras += n
        # facelib amplía internamente las imágenes de menos de 512 px de alto:
        # f es ese factor, para volver a las coordenadas del original.
        f = h.input_img.shape[0] / bgr.shape[0]
        pesos = [self._peso(float(b[2] - b[0]) / f) for b in h.det_faces]
        for p_ in pesos:
            self.stats["omitidas" if p_ <= 0 else "plenas" if p_ >= 1 else "parciales"] += 1
        if max(pesos) <= 0:
            return fondo_escalado
        h.align_warp_face()
        for cara, p_ in zip(h.cropped_faces, pesos):
            if p_ <= 0:
                h.add_restored_face(cara, cara)  # se descarta luego con peso 0
                continue
            t = self.img2tensor(cara / 255.0, bgr2rgb=True, float32=True)
            self.normalize(t, (0.5, 0.5, 0.5), (0.5, 0.5, 0.5), inplace=True)
            t = t.unsqueeze(0).to(self.device)
            with torch.no_grad():
                o = self.net(t, w=self.w, adain=True)[0]
            restaurada = self.tensor2img(o, rgb2bgr=True, min_max=(-1, 1)).astype("uint8")
            h.add_restored_face(restaurada, cara)
        h.get_inverse_affine(None)
        import cv2
        H, W = fondo_escalado.shape[:2]
        pegada = h.paste_faces_to_input_image(upsample_img=fondo_escalado.copy(), draw_box=False)
        if pegada.shape[:2] != (H, W):
            pegada = cv2.resize(pegada, (W, H), interpolation=cv2.INTER_AREA)
        pegada = pegada[:, :, :3].astype(np.uint8)
        # Mapa de mezcla: solo las zonas de cara salen de "pegada"; el resto del
        # plató queda exactamente como lo dejó Real-ESRGAN. Bordes suavizados.
        mapa = np.zeros((H, W), np.float32)
        s = self.escala / f
        for b, p_ in zip(h.det_faces, pesos):
            if p_ <= 0:
                continue
            x1, y1, x2, y2 = (float(v) * s for v in b[:4])
            mx, my = (x2 - x1) * 0.25, (y2 - y1) * 0.25
            xa, ya = int(max(x1 - mx, 0)), int(max(y1 - my, 0))
            xb, yb = int(min(x2 + mx, W)), int(min(y2 + my, H))
            mapa[ya:yb, xa:xb] = np.maximum(mapa[ya:yb, xa:xb], p_)
        k = int(max(H, W) * 0.01) | 1
        mapa = cv2.GaussianBlur(mapa, (k * 4 + 1, k * 4 + 1), 0)[..., None]
        out = fondo_escalado.astype(np.float32) * (1 - mapa) + pegada.astype(np.float32) * mapa
        return np.clip(out + 0.5, 0, 255).astype(np.uint8)
```

6.4. Interpolación a 50 fps y acabado (`acabado50.py`)

```
    filtros = ("eq=contrast=1.04:saturation=1.08,"
               "colorbalance=rm=0.03:bm=-0.03,"
               "scale=out_color_matrix=bt709:out_range=tv,format=yuv420p")
...
    with torch.inference_mode():
        while True:
            if a.max_fotogramas and n >= a.max_fotogramas:
                break
            cur = leer()
            if cur is None:
                break
            t_cur = tensor(cur, x0, x1)
            dif = (t_cur - t_prev).abs().mean().item()
            if dif > a.umbral_corte:
                medio = prev
                cortes += 1
            else:
                m = net(t_prev, t_cur, 0.5, escalas, False, True, False)
                m = m[:, :, :H, :x1 - x0].clamp(0, 1).mul(255).round().to(torch.uint8)
                medio = prev.copy()
                medio[:, x0:x1] = m[0].permute(1, 2, 0).cpu().numpy()
            encp.stdin.write(prev.tobytes())
            encp.stdin.write(np.ascontiguousarray(medio).tobytes())
            prev, t_prev = cur, t_cur
            n += 1
```

7. Resultados y limitaciones

7.1. Resultados cualitativos

Con el método D, los rostros en primer plano ganan definición en ojos, dientes y bordes sin cambiar su identidad visible. Las letras de cabeceras y rótulos mejoran con B, aunque no siempre con C. La interpolación a 50 fps es limpia en planos estables y en panorámicas lentas; se producen artefactos en movimientos muy rápidos y en las transiciones que no se detectan como corte.

7.2. Costes

Con el equipo utilizado, el informativo de 1992 completo (5289 fotogramas de salida a 25 fps, unos 3,5 minutos de vídeo) se procesó con el método D a 27,06 minutos de cálculo por minuto de vídeo. Con el acabado RIFE, el coste total del pipeline es de unos 38 minutos por minuto de vídeo, equivalente a 2 horas y 13 minutos para el vídeo completo de 3:31.

7.3. Limitaciones y evaluación pendiente

> **Evaluación objetiva pendiente.** Todavía no se han calculado métricas sin referencia (NIQE, MUSIQ, DOVER) ni una prueba de estabilidad de identidad facial sobre los tramos procesados. Las conclusiones de este informe son cualitativas, a partir de la inspección visual de fotogramas en `revision/`. Las métricas están previstas para la siguiente versión.

- **No hay recuperación de información perdida.** El material es 640×480 progresivo; todo detalle por encima de esa resolución es una estimación.
- **Riesgo de invención de detalle** en rótulos y letras pequeñas, especialmente con el modelo x4plus.
- **Coste:** el método D exige una GPU y varias horas para material largo. El modelo x4plus no compensa su coste en este material.
- **Caras en planos lejanos:** el control por tamaño reduce el problema, pero no lo elimina; algunos rostros pequeños quedan con el resultado de B.

8. Consideraciones de licencia y uso

CodeFormer tiene licencia S-Lab, que restringe el uso a fines no comerciales. Real-ESRGAN se distribuye con licencia BSD-3-Clause y RIFE con MIT. Antes de cualquier publicación o uso comercial debe revisarse la licencia de cada componente. Los materiales de Antena 3 son propiedad de su titular; este trabajo es de análisis técnico y conservación, y no sustituye los derechos de explotación.

9. Referencias bibliográficas

> Nota: las referencias se han recopilado con sus datos principales. Antes de cualquier envío a una revista conviene verificar páginas, DOI y versiones definitivas.

[1] Duchon, C. E. (1979). Lanczos filtering in one and two dimensions. *Journal of Applied Meteorology*, 18(8), 1016–1022.

[2] Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., & Loy, C. C. (2018). ESRGAN: Enhanced super-resolution generative adversarial networks. *Proceedings of the ECCV Workshops*.

[3] Wang, X., Xie, L., Dong, C., & Shan, Y. (2021). Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data. *Proceedings of the IEEE/CVF ICCV Workshops*, 1905–1914.

[4] Zhou, S., Chan, K. C. K., Li, C., & Loy, C. C. (2022). Towards robust blind face restoration with codebook lookup transformer. *Advances in Neural Information Processing Systems* (NeurIPS 2022).

[5] Huang, Z., Zhang, T., Heng, W., Shi, B., & Zhou, S. (2022). Real-time intermediate flow estimation for video frame interpolation. *Proceedings of the ECCV 2022*.

[6] Deng, J., Guo, J., Ververas, E., Kotsia, I., & Zafeiriou, S. (2020). RetinaFace: Single-shot multi-level face localisation in the wild. *Proceedings of the IEEE/CVF CVPR 2020*.

[7] Mittal, A., Soundararajan, R., & Bovik, A. C. (2013). Making a “completely blind” image quality analyzer. *IEEE Signal Processing Letters*, 20(3), 209–212.

[8] Ke, J., Wang, Q., Wang, Y., Milanfar, P., & Yang, F. (2021). MUSIQ: Multi-scale image quality transformer. *Proceedings of the IEEE/CVF ICCV 2021*.

[9] Wu, H., Zhang, E., Liao, L., Chen, C., Hou, J., Wang, A., Sun, W., Yan, Q., & Lin, W. (2023). Exploring video quality assessment on user-generated contents from aesthetic and technical perspectives (DOVER). *Proceedings of the IEEE/CVF ICCV 2023*.

[10] Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

[11] ITU-R (2015). *Recommendation BT.709-6: Parameter values for the HDTV standards for production and international programme exchange*.

[12] EBU (2023). *EBU R 128: Loudness normalisation and permitted maximum level of audio signals*.

[13] FFmpeg Developers. *FFmpeg documentation* (filtros `bwdif`, `cropdetect`, `idet`, `loudnorm`, `hevc_videotoolbox`). https://ffmpeg.org/documentation.html

[14] Apple Inc. *VideoToolbox framework* y *Metal Performance Shaders*, documentación para desarrolladores. https://developer.apple.com/documentation/

[15] PyTorch Team. *MPS backend* (aceleración Metal en Apple Silicon). https://pytorch.org/docs/stable/notes/mps.html

[16] Código de referencia utilizado: xinntao/Real-ESRGAN (licencia BSD-3-Clause); sczhou/CodeFormer (licencia S-Lab, **solo uso no comercial**); facelib y BasicSR (Wang, X. et al.); RIFE (licencia MIT, port de los pesos rife49.pth).

Anexo A. Código y repositorio

El código de este trabajo (`remaster4k.py`, `rife/acabado50.py`, `rife/rife_arch.py`, `filtro_original.py`, los lanzadores `.command` y el registro `registro_tiempos.csv`) está pendiente de publicarse en:

https://github.com/javierortizzamora/ia/tree/main/restauracion

Los pesos de los modelos (Real-ESRGAN, CodeFormer, RIFE) no se incluyen en el repositorio: se descargan de sus repositorios oficiales, cuyas URL aparecen en el propio código (diccionario `URLS`).

Anexo B. Registro de tiempos (extracto)

Fecha	Archivo	Modo	Fotogramas	Min. cálculo / min vídeo
30-10-2026	1992, t100–110 s	Lanczos	250	0,38
30-10-2026	1992, t100–110 s	General	250	5,83
30-10-2026	1992, t100–110 s	x4plus	250	194,33
30-10-2026	1992, t100–110 s	Caras (c40–90)	250	25,59
30-10-2026	1992, t143–153 s	General / Caras	250	8,14 / 35,73
30-10-2026	1992, t15–56 s	General / Caras	1025 / 500	7,88 / 23,07
01-11-2026	Intro 1989 (640×360)	x4plus	482	95,60
01-11-2026	1992 completo	Caras (c40–90)	5289	27,06
01-11-2026	Cortinilla 1992	Lanczos / General / x4plus	307	0,45 / 6,58 / 151,88
01-11-2026	1991, tramo 0–4 min	Caras (c40–90)	6000	25,27